

DaemonSet and StatefulSet

Class no:2

DaemonSet

In Kubernetes, a **DaemonSet** is a specific type of workload controller that ensures a single copy of a specific pod runs on **every single node** (or a chosen subset of nodes) in your cluster.

Think of it like a background system service or a "daemon" on a traditional Linux server (like sshd or cron), but containerized for a Kubernetes cluster. When a new node is added to your cluster, the DaemonSet automatically spins up a pod on it. If a node is removed, the pod is cleanly deleted.

Why use a DaemonSet? (Common Use Cases)

You typically use DaemonSets for background tasks that need to happen cluster-wide, usually falling into three categories:

1. **Log Collection:** Running a log forwarding agent on every node to collect logs and send them to a central system (e.g., Fluentd, Fluent Bit, or Logstash).
2. **Cluster Monitoring:** Running metrics collection agents on every node to monitor CPU, memory, and disk usage (e.g., Prometheus Node Exporter, Datadog agent, or New Relic).
3. **Cluster Networking:** Running the networking components required for nodes to talk to each other. (In fact, the **kube-proxy** and **Amazon VPC CNI** add-ons we talked about earlier run as DaemonSets behind the scenes!).

DaemonSet vs. Deployment: What's the difference?

While a **Deployment** is great for your regular applications, a DaemonSet is meant for infrastructure/management tools. Here is how they compare:

Feature	Deployment	DaemonSet
Pod Distribution	Pods are placed wherever there is space. A single node might get 3 pods, while another gets 0.	Exactly one pod per node (by default).
Scaling	You manually scale it (e.g., "give me 5 replicas") or use an autoscaler based on traffic.	Scales automatically with your cluster. 10 nodes = 10 pods. 100 nodes = 100 pods.
Typical Use Case	Web apps, APIs, microservices, stateless backends.	Monitoring agents, log collectors, network proxies.

StatefulSet

In Kubernetes, a **StatefulSet** is a workload controller designed to manage applications that **keep track of their state** (data).

While regular applications (like web servers or APIs) can be spun up, deleted, or replaced randomly without any issue, database-like applications need stability. They need to remember who they are, which disk belongs to them, and who they need to talk to. A StatefulSet provides exactly that stability.

The 3 Core Features of a StatefulSet

To understand why StatefulSets are unique, it helps to look at the three strict guarantees they provide to your pods:

1. Stable, Unique Network Identifiers

In a standard Deployment, pods get random names (e.g., frontend-7685b8fb-abcde). If a pod dies and a new one replaces it, the name changes completely.

- **StatefulSet Rule:** Pods are given a fixed, predictable, ordinal index starting from zero (e.g., mysql-0, mysql-1, mysql-2).
- **Why it matters:** If mysql-1 crashes, Kubernetes replaces it with a new pod named exactly mysql-1. Other parts of your system always know how to find it.

2. Stable, Persistent Storage

If a pod in a standard Deployment dies, any data written to its local disk is lost forever.

- **StatefulSet Rule:** It uses a feature called a **VolumeClaimTemplate**. This automatically provisions a unique cloud disk (like an Amazon EBS volume) for *each* individual pod slot.
- **Why it matters:** If mysql-2 crashes and gets recreated on a completely different EC2 worker node, Kubernetes automatically detaches the specific EBS volume that belonged to mysql-2 and re-attaches it to the new pod. Your data is completely safe and follows the pod identity.

3. Ordered, Graceful Deployment and Scaling

When you scale up a regular Deployment, Kubernetes throws all the new pods into the cluster at the exact same time.

- **StatefulSet Rule:** Pods are created one by one, in strict order (0, then 1, then 2). Pod 1 will not even attempt to start until Pod 0 is completely healthy and running. When scaling down, they are deleted in reverse order (2, then 1, then 0).
- **Why it matters:** Databases often require a "Primary" or "Master" node to exist before "Secondary" or "Replica" nodes can safely join the cluster and copy the data.

StatefulSet vs. Deployment

Feature 	Deployment	StatefulSet
Primary Use	Stateless apps (e.g., web frontends, APIs)	Stateful apps (e.g., databases, message brokers)
Pod Names	Random hash suffix (e.g., web-85c9b9-abcde)	Sequential and predictable (e.g., web-0, web-1)
Pod Identity	Pods are interchangeable	Pods have unique, sticky identities
Storage	Pods often share volumes or don't require data persistence	Each Pod gets its own exclusive persistent storage attached to its exact identity
Startup Order	All Pods start in parallel	Strictly ordered (0, 1, 2...)

Hands on: Creating DaemonSet and StatefulSet

Step no:1 Launch a **cluster** with the config we used previously and ensure to add: **Amazon EBS CSI Driver** in the add-ons

It will ask to **create a role**→ **create a new one and attach it**.

This will be needed in future for our stateful set.

Amazon EBS CSI Driver [Info](#)

Listed by



Category
storage

SI

Version

Select the version for this add-on.

v1.61.1-eksbuild.1

▼ Add-on access

Pod Identity IAM role for service account: ebs-csi-controller-sa

To access other AWS services and resources you need an IAM role. Choose a role with the right permissions.

AmazonEKSPodIdentityAmazonEBSCSIDriverRole successfully created.

AmazonEKSPodIdentityAmazonEBSCSIDriverRole



Create new role

[Edit role](#)

[View role details in IAM](#)

Step no:2 Create a node group with two worker nodes as the desired capacity.

Wait until it **is ready**.

mynodegp

Node group configuration [Info](#)

Kubernetes version
1.35

AMI release version | [Info](#)
1.35.5-20260529

AMI type | [Info](#)
Amazon Linux 2023 (x86_64) Standard

Instance types
c7i-flex.large

Status
 Active

Disk size
20 GiB

Once created we can see the **worker nodes** running on the **EC2 instance page**.

☐		i-08333940c9c35be30	Running	c7i-flex.large
☐	EKS	i-02a04addcdae9efa3	Stopped	t3.small
☐		i-0dc61580d505c1f1a	Running	c7i-flex.large

Step no:3 Launch our EKS instance and connect it with the worker nodes:

Aws eks update-kubeconfig --region our-respective-cluster-region --name cluster-name

```
Last login: Wed Jun 10 13:44:46 2026 from 13.233.177.3
[ec2-user@ip-172-31-8-71 ~]$ aws eks update-kubeconfig --region ap-south-1 --name mycluster
Updated context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster in /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-71 ~]$
```

i-02a04addcdae9efa3 (EKS)

PublicIPs: 13.200.229.70 PrivateIPs: 172.31.8.71

Check any existing pods: **kubectl get pods**

List the files: **ls** and remove all the existing ones: **rm ***

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pods
No resources found in default namespace.
[ec2-user@ip-172-31-8-71 ~]$ ls
deployment.yaml pod.yaml service.yaml services.yaml
[ec2-user@ip-172-31-8-71 ~]$ rm*
-bash: rm*: command not found
[ec2-user@ip-172-31-8-71 ~]$ rm *
[ec2-user@ip-172-31-8-71 ~]$ ls
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:3 In order to create a daemonset, create a **daemonset.yaml** file

vi daemonset.yaml

We can see the kind as DaemonSet

```
[ec2-user@ip-172-31-8-71 ~]$ vi daemonset.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: my-daemonset
  labels:
    app: my-daemon-app
spec:
  selector:
    matchLabels:
      app: my-daemon-app
  template:
    metadata:
      labels:
        app: my-daemon-app
    spec:
      containers:
      - name: nginx-container
        image: nginx:latest
        ports:
        - containerPort: 80
```

Apply the file using: **kubectl apply -f daemonset.yaml**

```
- containerPort: 80
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f daemonset.yaml
daemonset.apps/my-daemonset created
[ec2-user@ip-172-31-8-71 ~]$
```

To see the daemonset has been created list the pods: **kubectl get pods**.

As there are two worker nodes, two daemonset pods will be created.

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
my-daemonset-k9lc2                  1/1     Running   0           99s
my-daemonset-m78lk                  1/1     Running   0           99s
```

Step no:4 To expose the daemonset.yaml create a service for it.

The service must have same labels and selectors as that of the daemonset.yaml file.

vi service.yaml

cat service.yaml

we can see the selector and container port in both are same. Then only it will work. We are using LoadBalancer as the service type.

```
[ec2-user@ip-172-31-8-71 ~]$ vi service.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-daemon-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80          # Service port
      targetPort: 80    # Pod container port
```

Apply the service: **kubectl apply -f service.yaml**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:5 Go to the EC2 console. Open the worker node → security → inbound rule → Add All TCP enabled.

[EC2](#) > [Security Groups](#) > [sg-022063d73ed1ea5f4 - eks-cluster-sg-mycluster-1854704334](#) > [Edit inbound rules](#)

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

Security group rule ID	Type Info	Protocol Info	Port range Info	Source
sg-0e57eeb2e378588e5	All traffic ▼	All	All	Cluster
sg-0b160293ff2b64c83	All traffic ▼	All	All	Cluster
-	All TCP ▼	TCP	0 - 65535	Anywhere

[Add rule](#)

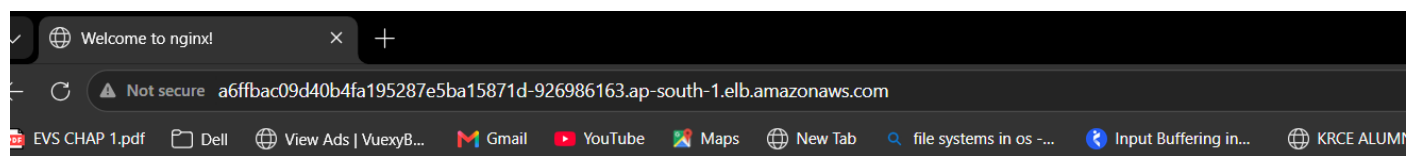
Go to load balancer in EC2 page → A **load balancer** has been already created by the service as it is of type load balancer.

Open it → **Copy the dns of the Load balancer** paste it in a private browser we can see the **output of the pod**.

✔ DNS name copied

[a6ffbac09d40b4fa195287e5ba15871d-926986163.ap-south-1.elb.amazonaws.com](#) (A Record)

[This Classic Load Balancer can be migrated to a next generation load balancer. Migration wizard uses y](#)



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Step no:6 Now we going to implement the statefulset. Go back to the EKS instance.

Hence create a **statefulset.yaml** file

```
vi statefulset.yaml
```

```
cat statefulset.yaml
```

we are creating a stateful set for MySQL as stateful sets are used for databases. It will have volume mounts to take backup of the image inside it to avoid losing the data. Its main aim is to be a persistent storage. That's why we have added a EBS add on in the cluster. They have environmental variables because in order to connect with a database we need a host name, user name and password.

```
[ec2-user@ip-172-31-8-71 ~]$ vi statefulset.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat s
service.yaml          statefulset.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat s
service.yaml          statefulset.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql-stateful
spec:
  serviceName: mysql-headless
  replicas: 2
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0

          ports:
            - containerPort: 3306

          env:
            - name: MYSQL_ROOT_PASSWORD
              value: root123
            - name: MYSQL_DATABASE
              value: mydb

          volumeMounts:
```

```
        - name: mysql-storage
          mountPath: /var/lib/mysql

      volumeClaimTemplates:
        - metadata:
            name: mysql-storage
          spec:
            storageClassName: gp2
            accessModes:
              - ReadWriteOnce
            resources:
              requests:
                storage: 2Gi
```

Apply the file using: **kubectl apply -f statefulset.yaml**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f statefulset.yaml
statefulset.apps/mysql-stateful created
```

To see the Statefulset has been created list the pods: **kubectl get pods.**

As there are two worker nodes, two Statefulset pods will be created.

```
service/mysql-headless created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
mysql-daemonset-k9lc2 1/1     Running   0          22m
mysql-daemonset-m78lk 1/1     Running   0          22m
mysql-stateful-0      1/1     Running   0          6m40s
mysql-stateful-1      1/1     Running   0          6m22s
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:7 To expose the statefulset.yaml create a service for it.

The service must have same labels and selectors as that of the statefulset.yaml file.

vi service-state.yaml

cat service-state.yaml

we can see the selector and container port in both are same. Then only it will work. We are using cluster ip as the service type. Because for databases we will not expose them over the internet. Hence the service type is the headless cluster ip. So there won't be any internet connection to the cluster Ip.

The kind: **StatefulSet**

In service type: **clusterIP: None**

```
[ec2-user@ip-172-31-8-71 ~]$ vi service-state.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat service-state.yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
    - port: 3306
      name: mysql
```

Apply the service: **kubectl apply -f service-state.yaml**

```
name: mysql
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f service-state.yaml
service/mysql-headless created
```


Step no:8 We can see the port number of **mysql:3306** as we have installed it via the pods.

The type as **ClusterIP with no external-ip**.

kubectl get svc

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	45m
my-app-service	LoadBalancer	10.100.130.242	a6ffbac09d40b4fa195287e5ba15871d-926986163.ap-south-1.elb.amazonaws.com	80:31135/TCP	21m
mysql-headless	ClusterIP	None	<none>	3306/TCP	118s

We can get inside of the pod (SQL Server): **kubectl exec -it mysql-stateful-0 --mysql -u root -p**

U stands for **username**. Here **root** is the username.

P stands for the **password** that we have set for the SQL database.

Enter the respective password. We will get inside the MySQL server.

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl exec -it mysql-stateful-0 -- mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql>
mysql>
```

To see the persistent volume content: **kubectl get pvc**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS
mysql-storage-mysql-stateful-0	Bound	pvc-70c4d4ec-a05a-416b-aca6-5e1fc888c41b	2Gi	RWO
mysql-storage-mysql-stateful-1	Bound	pvc-4dc92103-2f23-4bb7-abd5-0eff44b9be51	2Gi	RWO

```
[ec2-user@ip-172-31-8-71 ~]$
```

To see the stateful set: **kubectl get statefulset**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get statefulset
```

NAME	READY	AGE
mysql-stateful	2/2	10m

```
[ec2-user@ip-172-31-8-71 ~]$
```